2. Database Modeling

Exercise for the Challenge

Software Construction and Decision Making

Grupo 501

Daniela Janet Gil Gonzalez	A01752908
Yuhao Liu	A01787782
Mariana Ramirez Cervera	A01787819

28 de abril del 2026

The proposed relational scheme was designed to organize the main systems of the game: players, runs, maps, enemies, chests, decks, cards, math problems, and combat events. Since the game is a roguelike deck-building experience where each run changes and the player progresses through different maps, the database needs to store both permanent information, such as player progress, and temporary gameplay information, such as combats and selected cards. This structure supports the game mechanics described in the proposal, including deck building, changing maps, math-based combat, timer performance, enemies, bosses, chests, and rewards.

1. Player Table:

The Player table stores the basic information of each user, including username, password, HP, selected skin, win record, loss record, and creation date. This table is necessary because the game needs to identify each player and track their progress. It also allows the system to store statistics such as victories and defeats, which can be used to measure player improvement and replayability.

2. Game Table:

The Game table represents each individual run or match played by a player. It connects a player with a specific map and stores information such as start date, duration, and result. This table is useful because Math Smash is based on repeated runs, so each attempt must be recorded separately. It helps analyze how players perform in different maps and difficulties.

3. Map Table:

The Map table stores the different worlds or levels of the game, such as Ancient Temple, Castle, and Wasteland. It includes attributes like theme, difficulty, and math operation type. This table is important because each map changes the gameplay experience by introducing different enemies, challenges, and mathematical operations.

4. Enemy Table:

The Enemy table stores the enemies that appear in each map. It includes enemy name, type, HP, and attack power. This table allows the game to manage different enemy behaviors and difficulty levels. Since each map can contain multiple enemies, the relationship between Map and Enemy helps organize which enemies belong to each world.

5. Chest table:

The Chest table stores the chests that appear in the map. Chests can be normal rewards or traps, so the table includes type, HP in case it increases the player's points, and trap attack in case the chest triggers a surprise enemy or a math problem. This supports the random event system described in the game proposal.

6. Card Table:

The Card table stores all possible cards in the game, including attack, defense, and skill cards. Each card has a name, type, description, power value (defense or attack), and effect. This table is necessary because cards are one of the main mechanics of the game. It allows the system to define what each card does and how it affects combat.

7. Deck Table:

The Deck table stores the decks created or selected by each player. It includes the player ID, creation date, skill count for the restriction, last modification date, and quantity of cards. This table is necessary because players build decks before combat, choosing card types and credit levels. It also supports the rule that each deck can only include one skill card.

8. Math problem Table:

The Math Problem table stores the math exercises generated during a game. It includes difficulty, operation type, operands, and answer. This table is important because the main mechanic of the game depends on solving math problems to activate cards. It also allows the system to adjust difficulty depending on the map and track the type of operations used.

9. Combat deck Table:

The Combat Deck table is an intermediate table between Deck and Card. Since one deck can contain many cards and one card can appear in many decks, this table solves the many-to-many relationship. It also stores the quantity of each card and if the card is active during combat. This is useful for managing card availability, locked cards, and temporary effects from enemies.

10. Combat Table:

The Combat table records each combat event. It connects the game, enemy, card, and math problem used during the combat. It also stores the timer result, damage dealt, and combat result. This table is useful because it captures the most important gameplay interaction: the player selects a card, solves a problem, receives a timer score, and applies damage or defense.

Conclusion

This relational scheme supports the main logic of Math Smash: Card Adventure. It separates information into clear tables, avoids repetition, and connects the most important gameplay elements through relationships. The scheme can be used to save player progress, generate maps, manage enemies and chests, create decks, activate cards, generate math problems, and record combat results. This makes the database useful both for gameplay functionality and for future analysis of player performance.